

LoopCD: Loop-wise Contrastive Decoding for Improving Reasoning in Looped Language Models

Byeongho Yu^{1*} Junhyuk So^{1*} Eunhyeok Park²

¹Department of Computer Science and Engineering

²Graduate School of Artificial Intelligence

Pohang University of Science and Technology (POSTECH)

{bhyu418, junhyukso, eh.park}@postech.ac.kr

Abstract

Looped Language Models (LoopLMs) perform “latent reasoning” by recursively refining internal latent representations with shared weights, offering a more effective alternative to explicit verbal reasoning. Despite their effectiveness, we find that LoopLMs remain prone to *loop instability*: unstable refinement across iterations can produce localized uncertain “hard” tokens associated with reasoning errors. To address this, we propose **LoopCD**, loop-wise contrastive decoding that enhances the reasoning performance of LoopLMs by intervening on these tokens at inference time. Specifically, we exploit the internal dynamics of LoopLMs and contrast the logits from earlier iterations with logits from the last refined iteration to form the final sampling distribution. We find that this strategy is highly efficient, introducing only negligible inference overhead and requiring no additional training, while effectively improving reasoning performance by naturally refining reasoning-critical hard tokens. Extensive experiments show that our method improves the performance of recent representative LoopLMs across various reasoning tasks.

1 Introduction

Recent advances in large language models (LLMs) have shown that scaling both model size (Singh et al., 2026; DeepSeek-AI et al., 2025) and inference-time computation through *verbal thinking*, such as chain-of-thought (Wei et al., 2022), can substantially enhance reasoning capabilities. However, this paradigm can be inherently inefficient: models require massive parameter capacity to support different stages of inference (Lad et al., 2025), and verbalized thinking in discrete tokens imposes a linguistic bottleneck that leads to information collapse of rich continuous space (Hao et al., 2024).

As an emerging alternative, Looped Language Models (LoopLMs) (Geiping et al., 2025; Zhu

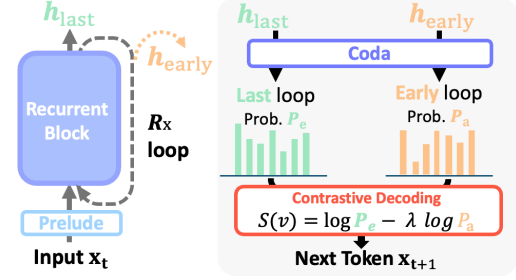


Figure 1: Overview of LoopCD at each decoding step.

et al., 2026) have been explored for *latent reasoning*, where models perform recursive computation directly in hidden space with shared weights, more naturally supporting different stages of inference and allowing models to reason in a richer continuous semantic space (Darlow et al., 2025). Recent studies have shown that this design yields strong parameter efficiency (Zhu et al., 2026) and strong performance across diverse reasoning tasks.

Despite their effectiveness, we observe *loop instability* in LoopLMs: for a small subset of tokens, additional latent iterations do not monotonically improve predictions, but can induce unstable dynamics such as premature confidence, failed revision, or erroneous refinements. Crucially, we find that this instability is often concentrated around reasoning-critical *hard* tokens, where localized errors can accumulate along the reasoning trajectory and eventually propagate to the final answer.

To mitigate this issue, we propose **LoopCD**,¹ a training-free inference-time decoding method for LoopLMs that substantially improves reasoning performance. Our key idea is to exploit the internal dynamics exposed by looped computation: while later iterations provide more refined predictions, earlier iterations retain complementary intermediate signals useful for contrast. By leveraging this loop-wise discrepancy, we construct the final sampling distribution by contrasting logits from earlier iterations with those from the final refined itera-

*These authors contributed equally.

¹<https://github.com/hoeng4/LoopCD>

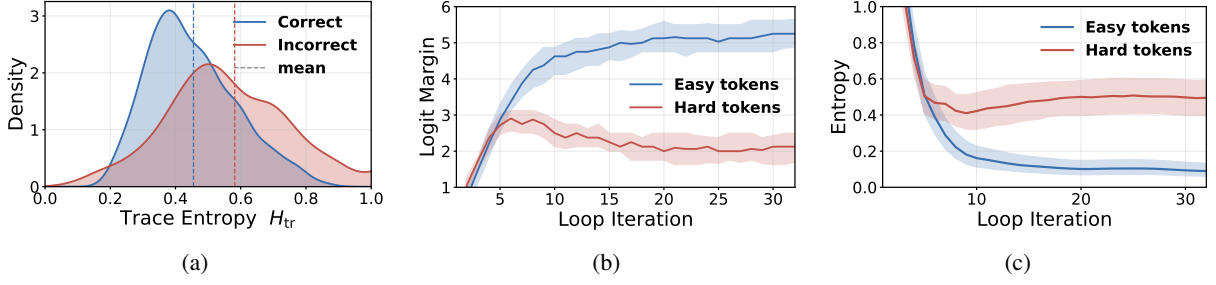


Figure 2: (a) Histogram of trace entropy H_{tr} for correct and incorrect GSM8K reasoning trajectories. (b, c) Logit margin and token entropy trajectories of easy and hard tokens across loop iterations.

tion. Our method requires neither auxiliary amateur models nor architectural modifications, and adds negligible computational overhead by reusing hidden states already produced during looping. Moreover, it naturally refines reasoning-critical hard tokens rather than the entire trajectory, achieving reliable reasoning. Extensive experiments show that LoopCD consistently improves LoopLM reasoning across arithmetic reasoning, code generation, and question answering benchmarks, outperforming standard and advanced decoding baselines.

2 Preliminaries

Looped Transformer. A Looped Transformer applies recurrence along the depth dimension by repeatedly reusing a fixed stack of Transformer blocks. We denote its architecture by $(p, k \otimes d, l)$, where p and l denote the numbers of layers in *prelude* and *coda*, corresponding to the layer stacks before and after the loop. k denotes the number of recurrent layers, and d is the recurrence depth. Let S_x denote an x -layer Transformer stack. $S'(\cdot)$ denotes untied Transformer stacks, while $S(\cdot)$ denotes weight-tied stacks across recurrences. Given input hidden states X , a simplified forward computation of Looped Transformer is

$$H = S'_l \left(\underbrace{(S_k \circ S_k \circ \dots \circ S_k)}_{d \text{ times}} (S'_p(X)) \right). \quad (1)$$

Thus, the model has parameter depth $p + k + l$ but realized computational depth $p + kd + l$. Finally, logits are produced after the last recurrence: $p(x_{i+1} | x_{\leq i}) = \text{softmax}(W_{\text{out}} H_i)$. Unless otherwise specified, verbalization is performed at a fixed maximum recurrence depth d_{max} shared by all tokens. We mainly consider the case of $(2, 4 \otimes 32, 2)$ Huginn-0125 (Geiping et al., 2025).

3 Motivation

Recent works on evaluating the reasoning performance of large language models have shown that

reasoning quality can be effectively estimated using the model’s internal signals (Fu et al., 2025; Yang et al., 2026; Kang et al., 2025). We also observe that the same approach can be safely applied to evaluating the reasoning performance of LoopLMs. We first define *trace entropy* H_{tr} to measure the model’s uncertainty along the reasoning trajectory.

$$H_{tr} = \frac{1}{N} \sum_{i=1}^N H_i, \quad H_i = - \sum_j p_i(j) \log p_i(j)$$

where i denotes the token position. In Fig. 2(a), we plot the histograms of H_{tr} for correct and incorrect reasoning trajectories on the arithmetic reasoning task GSM8K. As shown, incorrect reasoning trajectories generally exhibit higher entropy, whereas correct trajectories tend to have lower entropy. This suggests that, although LoopLMs are implicitly trained to reason in latent space, they can still produce uncertain tokens during verbalization, which substantially affects reasoning performance.

This naturally raises the question: which tokens in the decoding process of LoopLMs give rise to such uncertainty? To investigate this, we use logitlens (nostalgebraist, 2020) to examine how tokens are refined across different recurrence iterations. We observe two distinct patterns. As shown in Fig. 2(b), most tokens exhibit monotonic improvement behavior, where the margin between the top-1 and top-2 predictions increases over iterations. However, a minority subset of tokens exhibits the opposite behavior: the model becomes confident too early and later attempts to revise its initially confident prediction. We refer to the former case as *easy* tokens and the latter as *hard* tokens (formal definitions in Appendix C). As shown in Fig. 2(c), these hard tokens usually result in high-entropy distributions, and they account for about 10%–15% of the reasoning trace (Appendix Figure 4).

We observe that these hard tokens have a strong impact on reasoning quality. (1) As shown in Fig. 2(c), hard tokens generally lead to higher entropy

Correct→Incorrect	Easy Tokens	Hard Tokens
GSM8K	31.3%	52.9%
StrategyQA	25.9%	71.9%

Table 1: Percentage of originally correct trajectories that become incorrect after token replacement; accuracy is strict-match. Protocol details in Appendix D.

and therefore tend to contribute more to incorrect reasoning trajectories, as reflected in Fig. 2(a). (2) We further investigate the impact of hard tokens through random token replacement. As shown in Table 1, replacing hard tokens drops accuracy more substantially than replacing easy tokens, suggesting that the generated outputs are more sensitive to perturbations of hard tokens. This trend is further supported by the selective-intervention analysis in Section 5.4, where LoopCD’s gains are concentrated on hard tokens.

4 Method: LoopCD

Motivated by the observations above, we propose **LoopCD** to efficiently modulate the behavior of hard tokens during LoopLM inference. Our method is based on contrastive decoding (CD) (Li et al., 2023), but instead of using an external amateur model, it constructs the contrast internally from different iterations of looping. Specifically, at decoding step t , let $p_t^{(\ell)}(v)$ denote the probability assigned to token $v \in \mathcal{V}$ at loop iteration ℓ . We treat the distribution from a final iteration ℓ_e as the *expert* distribution, and the distribution from an earlier iteration $\ell_a < \ell_e$ as the *amateur* distribution. We compute a contrastive score as:

$$s_t(v) = \log p_t^{(\ell_e)}(v) - \lambda \log p_t^{(\ell_a)}(v), \quad (2)$$

and select the next token from the normalized distribution $p(v) = \frac{\exp(s_t(v))}{\sum_{u \in \mathcal{V}} \exp(s_t(u))}$. Here, λ is the contrastive coefficient, controlling how strongly the amateur distribution is penalized. We empirically choose the amateur loop index ℓ_a , as discussed in more detail in Section 5.3. To avoid promoting implausible tokens through contrastive scoring, we additionally apply the adaptive plausibility constraint described in Appendix B.1.

Mechanism. We now explain how our LoopCD improves reasoning by intervening on hard tokens from two perspectives. **i)** It serves as a corrective mechanism for early-overconfident predictions. If an earlier iteration assigns high probability to an incorrect token, subtracting the early amateur log probability suppresses this premature prefer-

ence and allows the later distribution to recover alternative tokens. **ii)** It can be viewed as a form of *contrasting guidance*, analogous to guidance commonly used in image generation. For example, diffusion models (Ho et al., 2020) often construct a score of the form $(1 + \alpha)\epsilon_\theta(x) - \alpha\epsilon_{\text{bad}}(x)$, where ϵ_{bad} represents an undesirable or less informative score, such as an unconditional score or under-trained model (Ho and Salimans, 2022; Karas et al., 2024). In LoopCD, the earlier loop iteration can naturally play the role of this less refined “bad” predictor. We provide further empirical evidence supporting this mechanism in Appendix F.

LoopCD primarily affects hard tokens. One direct approach would be to explicitly identify hard tokens and selectively intervene on them during decoding, like MTI (Yang et al., 2026). However, this requires tracking token confidence during iteration, and introduces an additional hyperparameter for *hardness* threshold. Here, we show that our LoopCD can naturally avoid these issues by having a stronger effect on confusing hard tokens.

Consider two candidate tokens y and z at decoding position t , and define the margin at loop iteration ℓ as $\Delta^{(\ell)}(y, z) = \log p_t^{(\ell)}(y) - \log p_t^{(\ell)}(z)$. Under LoopCD, the margin becomes:

$$(1 - \lambda) \underbrace{\Delta^{(\ell_e)}(y, z)}_{M_e} + \lambda \underbrace{(\Delta^{(\ell_e)}(y, z) - \Delta^{(\ell_a)}(y, z))}_{M_{CD}}.$$

For tokens exhibiting the typical easy token behavior in Fig. 2(b), the margin increases from the amateur to the expert iteration, yielding $M_{CD} > 0$, so LoopCD preserves the top-1 prediction. In contrast, hard tokens exhibit larger changes between the amateur and expert iterations due to peak-drop confidence dynamics, which can yield $(1 - \lambda)M_e + \lambda M_{CD} < 0$ and change the top-1 prediction. Our contrastive term is naturally much more influential for these hard tokens, allowing LoopCD to naturally refine hard tokens without explicitly detecting hard tokens or introducing an additional hyperparameter. We further investigate this implicit selectivity in Section 5.4.

5 Experiments

Models. We focus our experiments on two different pretrained LoopLMs: Huginn-0125 (Geiping et al., 2025) and Ouro-1.4B (Zhu et al., 2026), which was trained with substantially more compute and on a larger dataset.

Benchmarks. We evaluate our method on three

Model	Method	GSM8K		MATH-500		HumanEval		MBPP		StrQA	Rel. Tok/s
		Flex	Strict	Flex	Strict	HE	HE+	MBPP	MBPP+		
Huginn-0125	Greedy	33.21	23.12	13.60	12.20	24.39	20.73	40.74	33.60	54.02	1.00×
	MTI	33.51	24.11	15.20	13.40	25.00	20.73	42.06	35.71	54.19	0.93×
	Self-Eval	34.04	24.03	13.80	12.60	21.95	19.51	40.48	33.60	53.62	0.28×
	NoiseCD	33.36	22.06	14.20	13.40	23.78	20.73	42.59	35.45	53.14	0.98×
	Ours	36.62 +3.41	26.46 +3.34	15.60 +2.00	15.20 +3.00	30.49 +6.10	27.44 +6.71	42.06 +1.32	36.77 +3.17	55.24 +1.22	0.98×
Ouro-1.4B	Greedy	78.70	60.65	50.20	34.60	69.50	65.85	72.75	61.90	64.00	1.00×
	MTI	76.88	57.85	54.20	22.60	70.73	65.24	74.07	62.96	65.07	0.93×
	Self-Eval	77.06	54.66	46.80	27.80	74.39	69.51	69.31	58.73	56.86	0.25×
	NoiseCD	77.03	57.92	51.00	34.20	71.95	67.68	71.16	59.26	64.72	0.99×
	Ours	81.05 +2.35	64.06 +3.41	51.40 +1.20	37.40 +2.80	74.39 +4.89	70.12 +4.27	74.60 +1.85	62.96 +1.06	66.81 +2.81	0.99×

Table 2: Performance and inference-speed comparison for Huginn and Ouro. Tokens/s is measured on MATH-500. Flex/Strict: flexible/canonical matching; “+”: EvalPlus. See Appendix B.3 for details.

reasoning domains: (1) Mathematical reasoning on GSM8K (Cobbe et al., 2021) and MATH-500 (Hendrycks et al., 2021), (2) Code generation on HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), and (3) commonsense QA on StrategyQA (Geva et al., 2021). Further details are provided in Appendix B.3.

Baselines. We primarily compare our method with *greedy decoding*, which corresponds to the standard decoding of LoopLMs. We further compare our method with several advanced decoding baselines: **(a) MTI** (Yang et al., 2026) is the most closely related baseline to ours, which intervenes on high-entropy tokens using negative-prompted classifier-free guidance. **(b) Self-Eval** (Ren et al., 2023) samples multiple candidate reasoning trajectories and selects the final output through self-evaluation-based reranking, at the cost of increased computation. **(c) NoiseCD.** We also test a CD variant that injects noise into final-loop hidden states and contrasts the resulting logits, following common CD strategies in VLMs (Leng et al., 2024).

Implementation Details. LoopCD has two hyperparameters: CD coefficient λ and the amateur loop index ℓ_a . The expert loop index ℓ_e is fixed to the final recurrent iteration of each model, i.e., 32 for Huginn-0125 and 4 for Ouro-1.4B. For a fair comparison, we choose each baseline using the same protocol and report its best setting. Further details are provided in Appendices B.1 and B.2.

5.1 Main Results

As shown in Table 2, LoopCD shows the best overall performance compared to standard greedy decoding and other decoding variants. We summarize the main observations below: **(a) MTI.** Although MTI can improve over greedy decoding, it requires

Method	GSM8K	MATH-500	Tok/s
Greedy	33.21	13.60	5.09 (1.00×
Ours	36.62	15.60	4.99 (0.98×
EE	33.66	14.60	10.06 (1.98×
EE+ Ours	34.12	15.60	9.89 (1.94×

Table 3: Flex accuracy and throughput of LoopCD with Early Exit (EE) on Huginn.

Loop Idx.	4	8	12	16	20
GSM8K	35.41	36.39	34.42	33.51	32.90
HumanEval	26.83	30.49	26.22	24.39	25.00

Table 4: Amateur loop sensitivity on Huginn ($\lambda = 0.3$).

a sensitive entropy-threshold hyperparameter and an additional negative-condition KV-cache pass, reducing throughput. In contrast, LoopCD achieves better performance without such extra components, suggesting that earlier-loop logits provide a more suitable contrastive signal for LoopLMs. **(b) Self-Eval.** Compared with this method based on multiple parallel rollouts, which substantially reduces throughput, LoopCD achieves strong reasoning performance while remaining efficient. **(c) NoiseCD.** NoiseCD shows improvement in only limited cases. Unlike LoopCD, it also affects easy tokens due to hidden-state noise injection, which can limit its performance gains. This supports our motivation for intervening on reasoning-critical hard tokens and shows the effectiveness of logit-space contrasting.

5.2 Compatibility with Early Exit

We also combine LoopCD with early exit, which stops recurrence before the final loop when the hidden states change only marginally across consecutive iterations. As shown in Table 3, early exit substantially improves throughput by reducing recurrent computation. LoopCD can be combined with early exit to improve reasoning quality while

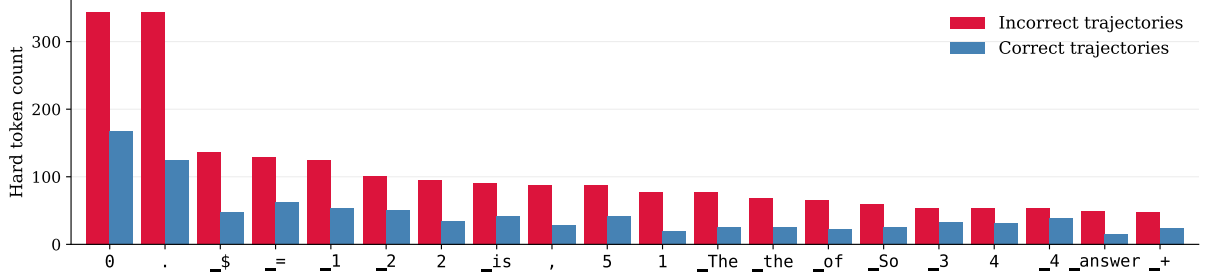


Figure 3: Top-20 hard token types from Huginn-0125 greedy generations on 600 GSM8K examples. Numerical tokens, arithmetic operators, punctuation, and reasoning-step markers appear frequently, suggesting that hard tokens are concentrated on tokens that affect intermediate computations and reasoning structure.

retaining the efficiency gains of early exit.

5.3 Amateur Loop Index Sensitivity

We study the sensitivity to the amateur loop index in LoopCD. As shown in Table 4, choosing an early loop, roughly 15–30% of the recurrence depth, often works well across tasks and models. This suggests that early representations are predictive enough while remaining distinct from the final iteration. We further analyze why this intermediate amateur range is particularly effective in Appendix E.3.

5.4 Ablation: Where Do LoopCD’s Gains Come From?

Due to their non-monotonic loop-wise dynamics, hard tokens are expected to be more strongly affected by LoopCD. We empirically examine this implicit selectivity and whether it accounts for LoopCD’s performance gains.

We first examine what kinds of tokens exhibit hard dynamics during reasoning. As shown in Figure 3, hard tokens frequently include numbers and arithmetic operators, which affect intermediate computations, as well as punctuation and reasoning-step markers such as The and So, which mark boundaries and transitions between reasoning steps. This suggests that loop instability is concentrated on tokens that shape intermediate quantities and the structure of the reasoning trajectory.

We next test whether hard tokens drive LoopCD’s gains by selectively applying it to hard or easy tokens. As shown in Table 5, hard-only intervention recovers much of the full gain despite affecting far fewer tokens, while easy-only intervention yields smaller improvements. This confirms that LoopCD’s gains are concentrated on hard tokens. The same trend holds within numeric and operator tokens, where hard subsets consistently outperform their easy counterparts. Nevertheless,

Setting	Acc. (%)	Δ	Interv. (%)
Greedy	33.21		0
LoopCD (full)	36.39	+3.18	100
Hard only	35.71	+2.50	10.7
Easy only	34.65	+1.44	89.0
Hard numeric only	34.04	+0.83	2.9
Easy numeric only	32.52	-0.68	26.2
Hard operator only	33.51	+0.30	0.7
Easy operator only	33.28	+0.08	6.9

Table 5: Performance of LoopCD under selective intervention on hard/easy tokens and numeric/operator tokens on Huginn ($\ell_a = 8$, $\lambda = 0.3$).

full LoopCD performs best, suggesting that explicitly restricting intervention to predefined token subsets can miss useful corrections. This supports our design of applying LoopCD to all tokens and relying on its implicit selectivity, rather than explicitly detecting hard tokens.

5.5 Further Analysis

We also present an extensive analysis of loop instability and the behavior of LoopCD (Appendix E), along with qualitative studies (Appendix H). To assess the robustness of LoopCD, we report results with fixed hyperparameter settings across benchmarks and multi-seed evaluations (Appendix G).

6 Conclusion

In this work, we showed that LoopLM reasoning trajectories contain reasoning-critical hard tokens whose non-monotonic loop-wise dynamics can substantially affect reasoning outcomes. To address this, we proposed LoopCD, an efficient inference-time method that contrasts earlier-loop logits with final-loop logits to effectively intervene on such tokens. Extensive experiments show that our LoopCD can consistently improve reasoning performance without additional training or architectural changes across diverse reasoning tasks.

Limitations

One limitation of LoopCD is that it relies on a training-free heuristic for selecting the contrastive signal. While this makes the method simple and efficient, the chosen amateur loop may not always provide an optimal contrast for every token, task, or model. We expect further improvements to be achievable by learning a verifier that identifies more reliable contrastive signals, or by adaptively selecting the optimal contrasting loop index during decoding.

Acknowledgments

This work was supported by IITP and NRF grant funded by the Korea government(MSIT) (RS-2026-25490269, RS-2019-II191906)

References

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. [Program synthesis with large language models](#). *Preprint*, arXiv:2108.07732.
- Shaojie Bai, J Zico Kolter, and Vladlen Koltun. 2019. Deep equilibrium models. *Advances in neural information processing systems*, 32.
- Shaojie Bai, Vladlen Koltun, and J Zico Kolter. 2020. Multiscale deep equilibrium models. *Advances in neural information processing systems*, 33:5238–5250.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. [Evaluating large language models trained on code](#). *Preprint*, arXiv:2107.03374.
- Yung-Sung Chuang, Yujia Xie, Hongyin Luo, Yoon Kim, James R Glass, and Pengcheng He. 2024. [Dola: Decoding by contrasting layers improves factuality in large language models](#). In *International Conference on Learning Representations*, volume 2024, pages 54158–54183.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. [Training verifiers to solve math word problems](#). *Preprint*, arXiv:2110.14168.
- Luke Darlow, Ciaran Regan, Sebastian Risi, Jeffrey Seely, and Llion Jones. 2025. [Continuous thought machines](#). In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 21548–21594. Curran Associates, Inc.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, and 181 others. 2025. [Deepseek-v3 technical report](#). *Preprint*, arXiv:2412.19437.
- Jeffrey L Elman. 1990. Finding structure in time. *Cognitive science*, 14(2):179–211.
- Yichao Fu, Xuewei Wang, Yuandong Tian, and Jiawei Zhao. 2025. Deep think with confidence. *arXiv preprint arXiv:2508.15260*.
- Zitian Gao, Lynx Chen, Yihao Xiao, He Xing, Ran Tao, Haoming Luo, Joey Zhou, and Bryan Dai. 2025. Universal reasoning model. *arXiv preprint arXiv:2512.14693*.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. [Scaling up test-time compute with latent reasoning: A recurrent depth approach](#). In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 41340–41391. Curran Associates, Inc.
- Ariel Gera, Roni Friedman, Ofir Arviv, Chulaka Gunasekara, Benjamin Sznajder, Noam Slonim, and Eyal Shnarch. 2023. The benefits of bad advice: Autocontrastive decoding across model layers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10406–10420.
- Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9:346–361.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2024. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851.
- Jonathan Ho and Tim Salimans. 2022. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*.

- Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation*, 9(8):1735–1780.
- Alexia Jolicoeur-Martineau. 2025. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*.
- Zhewei Kang, Xuandong Zhao, and Dawn Song. 2025. [Scalable best-of-n selection for large language models via self-certainty](#). In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 19720–19745. Curran Associates, Inc.
- Tero Karras, Miika Aittala, Tuomas Kynkäänniemi, Jaakko Lehtinen, Timo Aila, and Samuli Laine. 2024. Guiding a diffusion model with a bad version of itself. *Advances in Neural Information Processing Systems*, 37:52996–53021.
- Vedang Lad, Jin Hwa Lee, Wes Gurnee, and Max Tegmark. 2025. [Remarkable robustness of llms: Stages of inference?](#) In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 130050–130083. Curran Associates, Inc.
- Sicong Leng, Hang Zhang, Guanzheng Chen, Xin Li, Shijian Lu, Chunyan Miao, and Lidong Bing. 2024. Mitigating object hallucinations in large vision-language models through visual contrastive decoding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13872–13882.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 2022. [Solving quantitative reasoning problems with language models](#). *Preprint*, arXiv:2206.14858.
- Xiang Lisa Li, Ari Holtzman, Daniel Fried, Percy Liang, Jason Eisner, Tatsunori B Hashimoto, Luke Zettlemoyer, and Mike Lewis. 2023. Contrastive decoding: Open-ended text generation as optimization. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 12286–12312.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. [Let’s verify step by step](#). *Preprint*, arXiv:2305.20050.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and LINGMING ZHANG. 2023. [Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572. Curran Associates, Inc.
- Jieyi Long. 2023. Large language model guided tree-of-thought. *arXiv preprint arXiv:2305.08291*.
- Qing Lyu, Shreya Havaladar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. 2023. Faithful chain-of-thought reasoning. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 305–329.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. [s1: Simple test-time scaling](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20275–20321, Suzhou, China. Association for Computational Linguistics.
- nostalgebraist. 2020. Interpreting gpt: The logit lens. <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>. Accessed: 2026-05-26.
- Sean O’Brien and Mike Lewis. 2023. Contrastive decoding improves reasoning in large language models. *arXiv preprint arXiv:2309.09117*.
- Jie Ren, Yao Zhao, Tu Vu, Peter J. Liu, and Balaji Lakshminarayanan. 2023. [Self-evaluation improves selective generation in large language models](#). In *Proceedings on "I Can’t Believe It’s Not Better: Failure Modes in the Age of Foundation Models" at NeurIPS 2023 Workshops*, volume 239 of *Proceedings of Machine Learning Research*, pages 49–64. PMLR.
- Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, Akshay Nathan, Alan Luo, Alec Helyar, Aleksander Madry, Aleksandr Efremov, Aleksandra Spyra, Alex Baker-Whitcomb, Alex Beutel, Alex Karpenko, and 467 others. 2026. [Openai gpt-5 system card](#). *Preprint*, arXiv:2601.03267.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. 2020. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*.
- Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi-Yadkori. 2025. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*.
- Xintong Wang, Jingheng Pan, Liang Ding, and Chris Biemann. 2024. [Mitigating hallucinations in large vision-language models with instruction contrastive decoding](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 15840–15853, Bangkok, Thailand. Association for Computational Linguistics.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc.

Xumeng Wen, Zihan Liu, Shun Zheng, Shengyu Ye, Zhirong Wu, Yang Wang, Zhijian Xu, Xiao Liang, Junjie Li, Ziming Miao, Jiang Bian, and Mao Yang. 2025. [Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base llms](#). *Preprint*, arXiv:2506.14245.

Zhen Yang, Mingyang Zhang, Feng Chen, Ganggui Ding, Liang Hou, Xin Tao, and Ying-Cong Chen. 2026. [Less is more: Improving LLM reasoning with minimal test-time intervention](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 20124–20137, San Diego, California, United States. Association for Computational Linguistics.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822.

Byeongho Yu, Changhun Lee, Jun-gyu Jin, and Eunhyeok Park. 2025. [PruneCD: Contrasting pruned self model to improve decoding factuality](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 32462–32473, Suzhou, China. Association for Computational Linguistics.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, and 14 others. 2026. [Scaling latent reasoning via looped language models](#). *Preprint*, arXiv:2510.25741.

A Related Works

Reasoning in Language Models. With the development of large language models, research interest has gradually shifted from simple text generation to whether LLMs can perform reasoning required for real problem solving. A growing line of CoT-based reasoning methods (Wei et al., 2022; Muennighoff et al., 2025; Lyu et al., 2023) has shown the effectiveness of verbalizing intermediate reasoning steps for improving LLM reasoning. Following this line, later studies explored how models can learn to generate better CoT reasoning through training or reinforcement learning (Wen et al., 2025; DeepSeek-AI et al., 2025).

Another line of work has attempted to further improve reasoning through inference-time scaling. Tree of Thoughts (Yao et al., 2023; Long, 2023) explores multiple reasoning trajectories and selects more plausible paths through evaluation. Recent studies have further investigated whether reasoning quality can be estimated without a separate reward model, using signals from the model itself. Self-Consistency (Wang et al., 2022) shows that sampling multiple reasoning paths and selecting answers based on their agreement can serve as an effective inference-time strategy. More recent work such as DeepThink (Fu et al., 2025) shows that localized log-probability-based confidence within reasoning trajectories can effectively measure reasoning quality and enable more efficient test-time scaling.

Recurrence Models. Recurrence models, which repeatedly apply a fixed set of weights to refine representations in latent space, have been widely explored across many areas of machine learning. Early recurrent architectures, such as RNNs and LSTMs (Elman, 1990; Hochreiter and Schmidhuber, 1997), achieved strong results in time-series modeling and natural language processing by repeatedly applying shared parameters over sequential inputs. Deep Equilibrium Models (Bai et al., 2019, 2020) apply a shared transformation recurrently in latent space and directly solve for its fixed point, providing an implicit-depth architecture that can achieve competitive performance.

A major recent application of recurrence appears in generative modeling. Diffusion models (Ho et al., 2020; Song et al., 2020) generate data by iteratively applying a denoising network to progressively refine noisy variables, and have shown strong performance across image, text, and audio generation.

More recently, instead of relying on the standard sequential stack of distinct neural network layers, looped or recursive thinking models have attracted attention by repeatedly applying the same layer or module weights (Wang et al., 2025; Jolicoeur-Martineau, 2025; Gao et al., 2025). Recent studies find that such recurrent computation can achieve strong performance on reasoning tasks such as Sudoku and maze solving with far fewer parameters than conventional large neural networks.

Contrastive Decoding. Contrastive Decoding (CD) was originally proposed to improve the quality of open-ended text generation by contrasting an expert language model with a weaker amateur model, using their likelihood difference to reduce undesirable patterns such as repetition and incoherence (Li et al., 2023). Beyond its original setting, subsequent work has shown that contrastive decoding can improve reasoning (O’Brien and Lewis, 2023) and mitigate hallucination (Leng et al., 2024). A separate line of work constructs contrastive views from the same model using intermediate or early exit layers, model perturbations, or input distortion (Chuang et al., 2024; Gera et al., 2023; Yu et al., 2025; Wang et al., 2024). These methods suggest that CD does not require a separate model, but rather an informative negative view of undesirable or premature predictions. Recent work further shows that such intervention need not be applied uniformly across tokens. In particular, MTI (Yang et al., 2026) identifies high-entropy tokens as major sources of reasoning errors and applies test-time intervention only at such uncertain positions.

B Experimental Details

B.1 Implementation Details

Adaptive Plausibility Constraint. Contrastive decoding can assign a high score to tokens that have low probability under the amateur distribution, even when those tokens are unlikely under the expert distribution. To avoid selecting such implausible tokens, we follow prior work (Li et al., 2023) and adopt an Adaptive Plausibility Constraint.

$$\mathcal{V}_{\text{head}}(x_{<i}) = \{x_i \in \mathcal{V} : p_e(x_i | x_{<i}) \geq \alpha \max_{w \in \mathcal{V}} p_e(w | x_{<i})\}. \quad (3)$$

Only tokens in $\mathcal{V}_{\text{head}}(x_{<i})$ are considered when applying the CD objective. In all experiments, we set $\alpha = 0.1$, following the original setup. MTI and NoiseCD also employ the adaptive plausibility

constraint with $\alpha = 0.1$.

Deterministic Initialization. Huginn (Geiping et al., 2025) begins recurrent computation from an initial recurrent state, which is randomly initialized from a Gaussian distribution in the original model. For deterministic evaluation, we instead set this initial state to zero in our main experiments. We further verify robustness to stochastic initialization across multiple random seeds in Appendix G.2.

MTI (Yang et al., 2026). MTI intervenes only at uncertain decoding positions, using the model itself to construct a negative condition. At decoding step t , we take the distribution obtained after the full recurrence as $p_t^{\text{cond}} = p_t^{(\ell_e)}$, and regard the position as uncertain when its Shannon entropy exceeds a threshold, $H_t > \tau$.

$$H_t = - \sum_{v \in \mathcal{V}} p_t^{\text{cond}}(v) \log p_t^{\text{cond}}(v). \quad (4)$$

For such positions, we obtain an unconditional distribution by appending a fixed negative prompt c_{neg} to the prefix, and combine the two with classifier-free guidance:

$$s_t^{\text{MTI}}(v) = \begin{cases} s_t^{\text{CFG}}(v), & H_t > \tau, \\ \log p_t^{\text{cond}}(v), & \text{otherwise,} \end{cases} \quad (5)$$

$$s_t^{\text{CFG}}(v) = (1 - \omega) \log p_t^{\text{uc}}(v) + \omega \log p_t^{\text{cond}}(v). \quad (6)$$

The next token is selected as $\arg \max_v s_t^{\text{MTI}}(v)$. We use the original negative prompt $c_{\text{neg}} = \text{OUTPUT ERROR}$.

The official Hugging Face implementation deep-copies the KV cache before computing p_t^{uc} , which is expensive for LoopLMs due to their recurrent cache. We instead apply the negative prompt directly to the cache and truncate the appended entries afterwards. This yields the same p_t^{uc} without cache copying, so each intervention requires only one additional negative-prompt forward pass.

Self-Evaluation (Ren et al., 2023). For each problem x_i , we generate $N = 4$ candidate answers $\{y_{i,k}\}_{k=1}^N$ and select one using self-evaluation. The candidate generation procedure differs by model. For Ouro, we use stochastic decoding with temperature. For Huginn, we keep token decoding greedy and rely on randomized initial recurrent states to produce different candidates, since temperature-based token sampling degraded performance in our preliminary trials.

We then score each candidate with the same deterministic self-evaluation procedure. Given a candidate answer $y_{i,k}$, we construct a prompt asking whether the answer is correct:

Q: {question}
 Answer: {candidate answer}
 Is the above answer correct?
 A) Yes
 B) No
 Answer:

We run one deterministic forward pass and extract the logits of the single-token labels “A” and “B” at the final position. Let $z_A(y_{i,k})$ and $z_B(y_{i,k})$ denote these logits. We define the self-evaluation score as

$$s(y_{i,k}) = z_A(y_{i,k}) - z_B(y_{i,k}), \quad (7)$$

and select the candidate with the highest score:

$$y_i^* = y_{i,k^*}, \quad k^* = \arg \max_k s(y_{i,k}). \quad (8)$$

NoiseCD. To test whether the improvement of LoopCD comes from the structured recurrent trajectory or merely from contrasting against a corrupted distribution, we introduce a noisy-amateur contrastive decoding baseline. Instead of using an earlier loop iteration as the amateur distribution, this baseline constructs an artificial amateur by perturbing the final hidden state with Gaussian noise.

Let $h_t^{(\ell_e)} \in \mathbb{R}^d$ denote the final expert hidden state at decoding position t . We construct a corrupted hidden state as

$$h_{t,\text{noise}} = h_t^{(\ell_e)} + \sigma \cdot \text{RMS}(h_t^{(\ell_e)}) \cdot \epsilon_t \quad (9)$$

$$\epsilon_t \sim \mathcal{N}(0, I),$$

where σ controls the relative noise strength. The RMS is computed over the hidden dimension:

$$\text{RMS}(h) = \sqrt{\frac{1}{d} \sum_{j=1}^d h_j^2}. \quad (10)$$

We then obtain the noisy-amateur distribution by applying the same language-model head to the corrupted hidden state:

$$p_t^{(\text{noise})}(v) = \text{softmax}(g_\theta(h_{t,\text{noise}}))_v. \quad (11)$$

Finally, we apply the same contrastive decoding rule as LoopCD:

$$s_t^{\text{NoiseCD}}(v) = \log p_t^{(\ell_e)}(v) - \lambda \log p_t^{(\text{noise})}(v). \quad (12)$$

This baseline removes the temporal structure of the recurrent trajectory while preserving a comparable corrupted-amateur setup. Therefore, com-

paring LoopCD against NoiseCD isolates whether the benefit comes from the earlier loop iteration itself, rather than from subtracting an arbitrary noisy distribution.

Early Exit. LoopLMs compute by repeatedly applying a recurrent block to refine latent hidden states. However, once the hidden state changes only marginally between consecutive recurrent steps, the remaining iterations may provide little additional computation for the current token. This observation motivates early exit, which terminates the recurrent loop when the latent representation has sufficiently stabilized.

For Huginn, we follow the early exit criterion described in the original paper (Geiping et al., 2025). At recurrent iteration r , we compute the normalized change of the hidden state between two consecutive iterations:

$$d_r = \frac{1}{T} \sum_{j=1}^T \frac{\|h_{r,j} - h_{r-1,j}\|_2}{\|h_{r,j}\|_2}. \quad (13)$$

We exit at the first recurrent iteration satisfying $d_r < \tau$. We set $\tau = 0.03$. When combining early exit with LoopCD, we allow early exit only after 15 recurrent iterations and use the early exit iteration as the expert loop. We separately search the amateur loop index for the combined setting.

B.2 Hyperparameters

We describe the hyperparameters considered for LoopCD and each baseline. For fair comparison, we use the best-performing setting for each method in our reported results.

LoopCD. LoopCD has two hyperparameters: the amateur loop index ℓ_a and the CD coefficient λ . The amateur loop index selects the intermediate loop output used as the amateur distribution. Very early iterations can be noisy, while later iterations can be too close to the expert distribution, weakening the contrastive signal. We therefore search ℓ_a over $\{6, 8, 10\}$ for Huginn and $\{0, 1\}$ for Ouro. For the contrastive strength, we search $\lambda \in \{0.1, 0.2, 0.3\}$ for both models.

MTI. MTI has two hyperparameters: the guidance scale and the entropy threshold. For the guidance scale, we follow the original MTI setting and set it to 1.5. Since the entropy threshold can vary across models and benchmarks, we search it over $\{0.5, 1.0, 1.5\}$ for both Huginn and Ouro.

Self-Evaluation. For Ouro, we use temperature sampling with nucleus sampling. In addition to the original Ouro decoding setting ($T = 1.0, p = 0.7$), we also evaluate ($T = 0.7, p = 0.9$) and report the better performing configuration.

NoiseCD. NoiseCD uses the same contrastive coefficient λ as LoopCD, and we fix $\lambda = 0.3$ in all experiments. Its additional hyperparameter is the noise scale σ , which controls the strength of the Gaussian perturbation added to the expert hidden state. We set $\sigma = 0.5$ as a perturbation level.

B.3 Benchmark Details

GSM8K. We evaluate on the official GSM8K test split of 1,319 problems using the 3-shot prompt. We report both Strict and Flex accuracy. Strict requires the canonical final-answer format, while Flex extracts the final numerical answer from the completion.

MATH-500. We evaluate on MATH-500, a 500-problem subset of MATH (Hendrycks et al., 2021), introduced by Lightman et al. (2023), using the 4-shot Minerva prompt (Lewkowycz et al., 2022). We report Strict accuracy with the Minerva-MATH evaluator and Flex accuracy with `math_verify`, which accepts mathematically equivalent answer forms.

HumanEval / HumanEval+. We evaluate HumanEval, a Python code-generation benchmark consisting of 164 function-completion problems, and HumanEval+, its EvalPlus-augmented version (Liu et al., 2023) with additional test cases. We use zero-shot greedy decoding with the original function signature and docstring as the prompt. Following EvalPlus, we report both base-test and plus-test results. In our tables, **HE** and **HE+** denote the base and plus test results, respectively.

MBPP / MBPP+. We evaluate MBPP, a Python code-generation benchmark consisting of short natural-language programming tasks, and MBPP+, its EvalPlus-augmented version with additional test cases. We use the sanitized 378-problem MBPP+ split and perform zero-shot greedy decoding. Following EvalPlus, we report both base-test and plus-test results. In our tables, **MBPP** and **MBPP+** denote the base and plus test results, respectively.

StrategyQA. We evaluate StrategyQA, a binary commonsense QA benchmark requiring implicit multi-step reasoning. We use the 2,290-question

training split because the official test labels are hidden. We use a 6-shot Chain-of-Thought prompt and greedy decoding. We extract the final yes/no prediction and report accuracy.

C Formal Definition of Hard Tokens

For our Huginn-0125 analyses, we define hard tokens as follows. For each generated position, let $z_i(v)$ denote the logit assigned to token v at recurrent iteration i , and let

$$y^* = \arg \max_{v \in \mathcal{V}} z_{32}(v)$$

be the final winner token at the last recurrent iteration. We define the fixed-target logit margin of y^* at iteration i as

$$m_i = z_i(y^*) - \max_{v \in \mathcal{V}, v \neq y^*} z_i(v).$$

The peak margin and final margin are then

$$m_{\text{peak}} = \max_i m_i, \quad m_{\text{final}} = m_{32}.$$

We define the confidence drop of the final winner as

$$d = m_{\text{peak}} - m_{\text{final}}.$$

A generated token is classified as hard if

$$d \geq \epsilon,$$

and as easy otherwise. We set $\epsilon = 1.5$ throughout our experiments, under which approximately 89.3% of generated tokens are classified as easy.

D Token replacement protocol

For each example correctly answered under greedy decoding, we sample $k = \min(5, |\text{HARD}|, |\text{EASY}|)$ positions from either hard or easy tokens and replace each with a token sampled uniformly from the top-3 candidates at that position, while keeping the rest of the completion fixed. We use the same replacement budget for both classes within each example to ensure a comparable setting. Examples lacking either class are excluded, leaving $n = 303$ for GSM8K and $n = 551$ for StrategyQA.

We evaluate the perturbed completions using strict-match accuracy based on the benchmark-specific answer format. Under this protocol, perturbing hard tokens causes a substantially larger accuracy drop than perturbing easy tokens, showing that the generated outputs are more sensitive to perturbations of hard tokens. This trend is further supported by the selective-intervention analysis in Section 5.4, where LoopCD’s gains are concentrated on hard tokens.

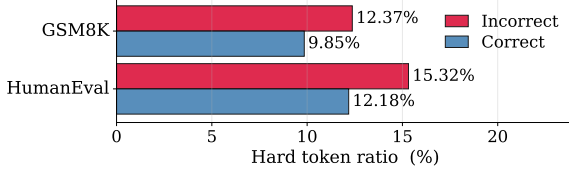


Figure 4: Proportion of hard tokens in correct and incorrect reasoning trajectories. Hard tokens account for roughly 10-15% of generated tokens and are more prevalent in incorrect trajectories.

Metric	Easy Tokens	Hard Tokens	Ratio
Margin Flip	0.82%	8.78%	10.7×

Table 6: Margin-flip rates for easy and hard tokens on Huginn with GSM8K ($\ell_a = 8$, $\lambda = 0.3$).

E Analysis of Loop Instability and LoopCD Behavior

We provide additional analyses to complement the ablation in Section 5.4 and better understand why LoopCD’s gains are concentrated on hard tokens. We first examine whether hard tokens are more prevalent in incorrect reasoning trajectories, then quantify whether LoopCD naturally affects hard tokens more strongly than easy tokens, and finally analyze why early recurrent iterations provide effective amateur distributions.

E.1 Finding 1: Hard Tokens Are More Prevalent in Incorrect Trajectories

We first examine whether hard tokens are associated with unsuccessful reasoning trajectories. As shown in Figure 4, incorrect generations consistently contain a higher proportion of hard tokens than correct generations on both GSM8K and HumanEval. This suggests that loop instability is more prevalent in failed reasoning trajectories, further highlighting the importance of hard tokens in reasoning failures.

E.2 Finding 2: LoopCD Disproportionately Affects Hard Tokens

Given that hard tokens are more prevalent in wrong reasoning trajectories, we next examine whether LoopCD naturally affects these tokens more strongly than easy tokens. For each generated token t , let

$$y_t^* = \arg \max_v z_{32,t}(v)$$

be the greedy prediction at the final recurrent iteration. Given an amateur iteration ℓ_a and contrastive weight λ , we define the LoopCD contrastive logit

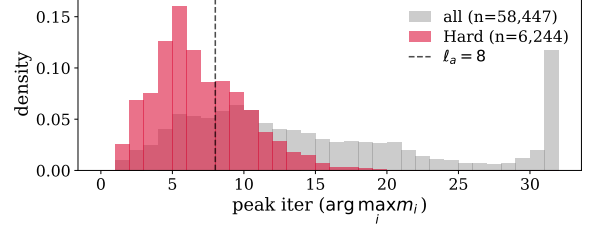


Figure 5: Peak iteration of the fixed-target margin $\arg \max_i m_{t,i}$ for hard and all tokens.

as

$$\tilde{z}_t(v) = z_{32,t}(v) - \lambda z_{\ell_a,t}(v).$$

We say that token t is *margin-flipped* if the contrastive logit changes the original greedy prediction:

$$\tilde{z}_t(y_t^*) < \max_{v \neq y_t^*} \tilde{z}_t(v).$$

This diagnostic measures whether the LoopCD contrast is strong enough to alter the final greedy decision without explicitly identifying hard tokens during decoding.

Result. As shown in Table 6, among 58,447 generated tokens, hard tokens ($N = 6,244$) are margin-flipped at a rate of 8.78%, whereas easy tokens ($N = 52,203$) are flipped at only 0.82%. This corresponds to a $10.7\times$ higher margin-flip rate for hard tokens, showing that although LoopCD is applied to all tokens, its contrastive effect naturally concentrates on hard tokens.

E.3 Finding 3: Hard Tokens Peak Near the Amateur Iteration

We next investigate why an intermediate recurrent iteration, around 15–30% of the recurrence depth, provides an effective amateur distribution. For each generated token t , we track the fixed-target margin of the final greedy prediction across recurrent iterations:

$$m_{t,i} = z_{i,t}(y_t^*) - \max_{v \neq y_t^*} z_{i,t}(v),$$

and record the iteration where this margin is maximized, $\arg \max_i m_{t,i}$.

As shown in Figure 5, hard tokens concentrate sharply around iterations 6-12, with a mode near $\ell_a = 8$. In contrast, the full token population, which is dominated by easy tokens, is more broadly distributed and shifts toward later iterations. This suggests that $\ell_a = 8$ is not merely an arbitrary hyperparameter: it captures the transient overconfidence of hard tokens, where the amateur-expert contrast becomes most effective. This observation is also

Type	Correct	Wrong
Early Overconfidence	9.4%	11.1%
Late-Flip	2.45%	2.96%

Table 7: Frequency of early overconfidence and Late-Flip positions in Huginn GSM8K greedy generations.

consistent with the performance sweep in the main text, where amateur iterations around $\ell_a \approx 8$ work best.

F Expert/Amateur Mechanism and Failure Mode Distinction

F.1 Justification of the Final-Loop Expert

In LoopCD, “expert” and “amateur” are relative terms inherited from contrastive decoding and do not imply guaranteed correctness. We use the final iteration as the expert because it is the distribution from which the original LoopLM verbalizes, while an earlier iteration provides the contrastive signal. LoopCD therefore amplifies the early-to-final update direction, which improves reasoning on average, but may reinforce an error when later iterations move away from a better earlier prediction.

We distinguish this failure mode from early overconfidence by analyzing Huginn-0125 on GSM8K. We define a *Late-Flip* position as one where the final-loop winner is not top-1 at least once during the latter half of the recurrent iterations. We define an early-overconfidence position as one satisfying the same confidence-drop criterion used to define hard tokens in Appendix C, while its final-loop winner remains top-1 throughout the latter half of the recurrence (i.e., it is not a Late-Flip). Since a Late-Flip can be beneficial or harmful, its frequency provides an upper bound on potential late-loop erroneous revisions.

As shown in Table 7, Late-Flips are substantially less frequent than early-overconfidence positions, occurring at only about 2–3% of generated tokens. To examine whether these flips actually correspond to harmful revisions, we replace the final-loop winner at each Late-Flip position with the most frequent alternative top-1 token over iterations 17–32, excluding the final-loop winner, and resume greedy decoding.

As shown in Table 8, this replacement can repair late-loop errors, but more often disrupts an otherwise correct trajectory. Thus, late-loop erroneous revisions do occur, but beneficial late revisions are

Outcome	Rate
Incorrect $\rightarrow$ Correct	9.9%
Correct $\rightarrow$ Incorrect	23.9%

Table 8: Effect of replacing Late-Flip positions with alternative top-1 predictions.

more common, supporting the use of the final loop as the expert anchor on average.

F.2 Representative Failure Cases

We next examine failures introduced by LoopCD itself, where Greedy decoding would otherwise produce a correct trajectory.

CASE 1: Unnecessary intervention on an easy token. Although LoopCD disproportionately affects hard tokens, its implicit selectivity is not perfect. For example:

Greedy [Correct]: ... $43 \times 3 = 129$...

LoopCD [Incorrect]: ... $43 \times 3 = 1,293$...

At the divergent position, both the expert and amateur prefer the correct token 2, but the amateur assigns it slightly higher confidence. The contrastive term therefore over-penalizes 2 and promotes the comma token “,”, thereby overturning an otherwise correct easy token prediction. Consistent with Appendix E.2, such easy token margin flips are rare (0.82%).

CASE 2: Initial-token trajectory divergence.

LoopCD can also change the initial generated token and thereby redirect the subsequent reasoning trajectory:

Greedy [Correct]:

"The original piece of wire was 4 feet long ..."

LoopCD [Incorrect]:

"4 feet is $4 \times 12 = 48$ inches ..."

Here, the initial token changes from The to 4. Because subsequent predictions are conditioned on the generated prefix, this early intervention leads to a different reasoning path and ultimately an incorrect answer. Such initial-token divergence occurs in approximately 1.2% of the evaluated trajectories.

Overall, beneficial corrections remain more common: on GSM8K with Huginn, the ratio of incorrect-to-correct changes to correct-to-incorrect changes is 1.49.

G Robustness of LoopCD

G.1 Fixed-Hyperparameter Results

We evaluate whether LoopCD remains effective without benchmark-specific tuning. For each

Model	Method	GSM8K		MATH-500		HumanEval		MBPP		StrQA
		Flex	Strict	Flex	Strict	HE	HE+	MBPP	MBPP+	
Huginn-0125	Greedy	33.21	23.12	13.60	12.20	24.39	20.73	40.74	33.60	54.02
	Ours	36.39 +3.18	24.34 +1.22	15.60 +2.00	14.20 +2.00	30.49 +6.10	27.44 +6.71	42.06 +1.32	36.77 +3.17	55.24 +1.22
Ouro-1.4B	Greedy	78.70	60.65	50.20	34.60	69.50	65.85	72.75	61.90	64.00
	Ours	79.83 +1.13	62.40 +1.75	51.20 +1.00	37.20 +2.60	71.34 +1.84	67.68 +1.83	72.75 +0.00	62.17 +0.27	64.67 +0.67

Table 9: Performance comparison for Huginn and Ouro with fixed hyperparameters.

Method	GSM8K	MATH-500	HE	HE+	MBPP	MBPP+	StrQA
Greedy	32.98 $\pm$ 0.97	13.75 $\pm$ 0.71	23.32 $\pm$ 1.46	19.66 $\pm$ 1.46	40.67 $\pm$ 0.63	33.33 $\pm$ 0.60	53.77 $\pm$ 0.07
LoopCD	35.88 $\pm$ 0.72 +2.90	16.00 $\pm$ 1.58 +2.25	30.49 $\pm$ 1.37 +7.17	27.59 $\pm$ 1.46 +7.93	42.79 $\pm$ 1.05 +2.12	35.32 $\pm$ 1.00 +1.99	55.71 $\pm$ 0.68 +1.94

Table 10: Performance of Huginn-0125 with LoopCD averaged over four seeds. Results are mean accuracy (%) $\pm$ 95% confidence intervals. GSM8K and MATH-500 use flex accuracy.

model, we use one fixed configuration across all benchmarks: $\ell_a = 8$ and $\lambda = 0.3$ for Huginn, and $\ell_a = 1$ and $\lambda = 0.2$ for Ouro.

As shown in Table 9, LoopCD improves over Greedy decoding on all benchmarks except MBPP with Ouro, where it matches Greedy. This suggests that LoopCD is robust to fixed, benchmark-agnostic hyperparameter settings.

G.2 Multi-Seed Evaluation

To further assess the robustness of LoopCD, we evaluate Huginn across four random seeds. Huginn initializes its recurrent state with Gaussian noise, so different seeds induce different latent trajectories even under greedy decoding. Table 10 reports the mean accuracy and 95% confidence intervals across the four seeds. LoopCD improves the mean performance over Greedy across all benchmarks. These results show that the improvements of LoopCD are robust to variations in the randomized initial recurrent state.

H Qualitative Examples

We present qualitative examples comparing Greedy decoding and LoopCD in Figure 6. The highlighted tokens indicate positions where the two decoding methods diverge. These examples show that LoopCD often modifies reasoning-critical tokens, leading to improved reasoning outcomes.

Q: Ducks need to eat 3.5 pounds of insects each week to survive.
If there is a flock of ten ducks, how many pounds of insects do they need per day?

Greedy [Incorrect]

A. Ducks need to eat 3.5 pounds of insects each week. If there is a flock of ten ducks, then they need $3.5 \times 10 = 35$ pounds of insects per week.
If there are 7 days in a week, then they need $35 \times 7 = 245$ pounds of insects per day.
The answer is 245.

LoopCD [Correct]

A. Ducks need to eat 3.5 pounds of insects each week. If there is a flock of ten ducks, then they need $3.5 \times 10 = 35$ pounds of insects per week.
If there are 7 days in a week, then they need $35 / 7 = 5$ pounds of insects per day.
The answer is 5.

Q: The temperature was 2 degrees Celsius.
It dropped 8 degrees Celsius overnight and then increased 3 degrees Celsius in the morning.
What was the temperature, in degrees Celsius, in the morning?

Greedy [Incorrect]

A. The temperature was originally 2 degrees Celsius. It dropped 8 degrees Celsius overnight.
So the temperature in the morning was $2 + 8 = 10$ degrees Celsius.
Then it increased 3 degrees Celsius in the morning.
So the temperature in the morning was $10 + 3 = 13$ degrees Celsius. The answer is 13.

LoopCD [Correct]

A. The temperature was originally 2 degrees Celsius. It dropped 8 degrees Celsius overnight.
So the temperature in the morning was $2 - 8 = -6$.
Then it increased 3 degrees Celsius in the morning.
So the temperature in the morning was $-6 + 3 = -3$. The answer is -3.

Figure 6: Qualitative comparison between Greedy decoding and LoopCD. Tokens highlighted in red and green indicate positions where the generations of Greedy decoding and LoopCD diverge. LoopCD adjusts these reasoning-critical tokens, leading to more consistent reasoning trajectories and correct final answers.